

Kubernetes DaemonSet

VISHWANATH M S
[VISHWACLOUDLAB.COM](https://vishwacloudlab.com)

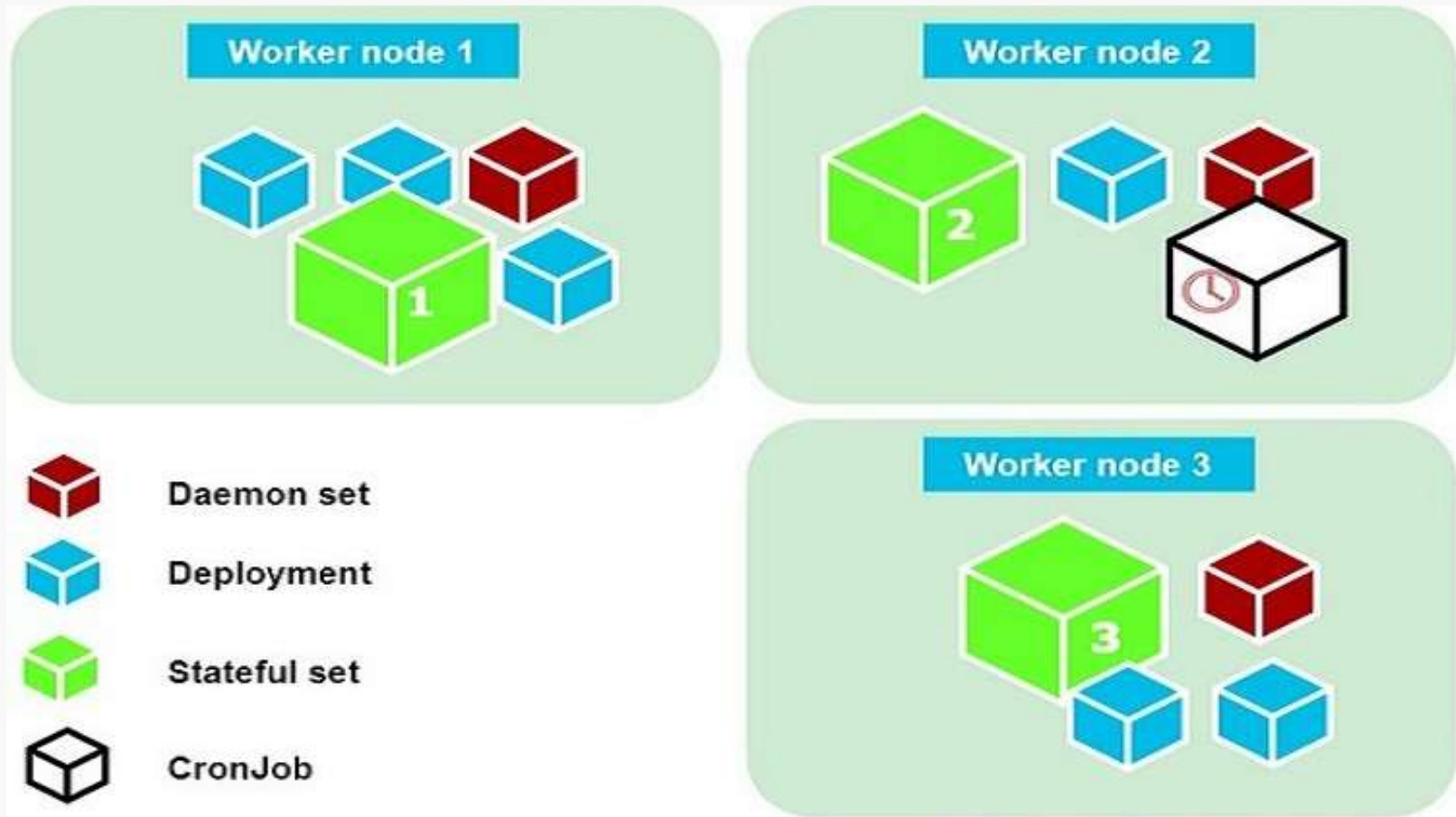


Agenda

- What is DaemonSet
- How it works
- Example
- Update strategies
- Common Use Cases
- Potential Challenges



Kubernetes DaemonSet





What Is a DaemonSet?

Definition: A Kubernetes control that ensures a copy of a pod runs on all—or a subset of—nodes

Typical use: deploying monitoring/logging agents

Visual: Diagram showing pods per node

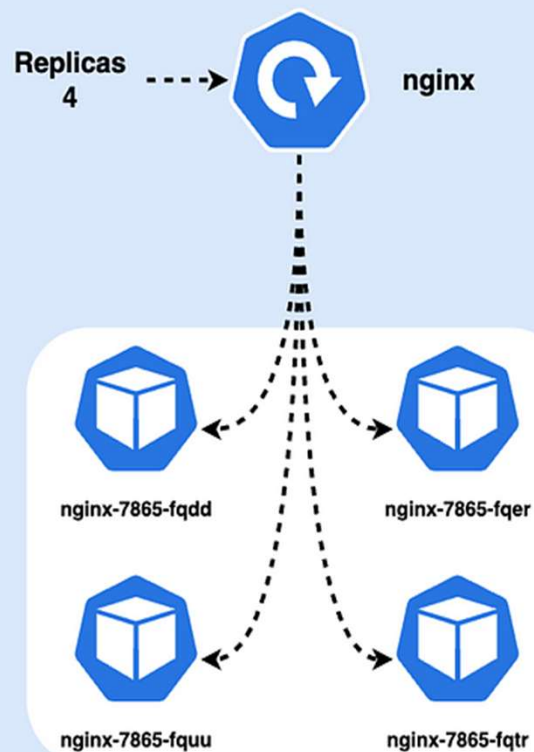


DaemonSet vs. Other Controllers

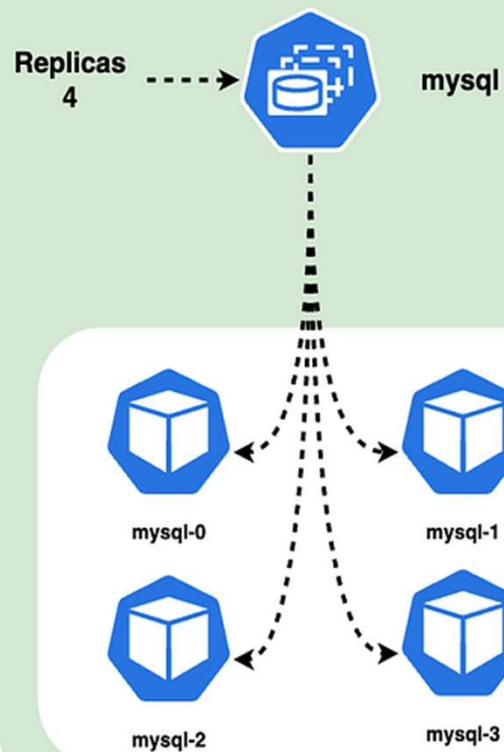
Controller	Purpose	Common Use Cases
DaemonSet	One pod per node	Node-level logging, security, observability
Deployment	Pods placed arbitrarily across cluster	Standard stateless application workloads
StatefulSet	Pods with unique, persistent identities	Stateful apps requiring individual storage

Deployments, StatefulSets, and DaemonSets

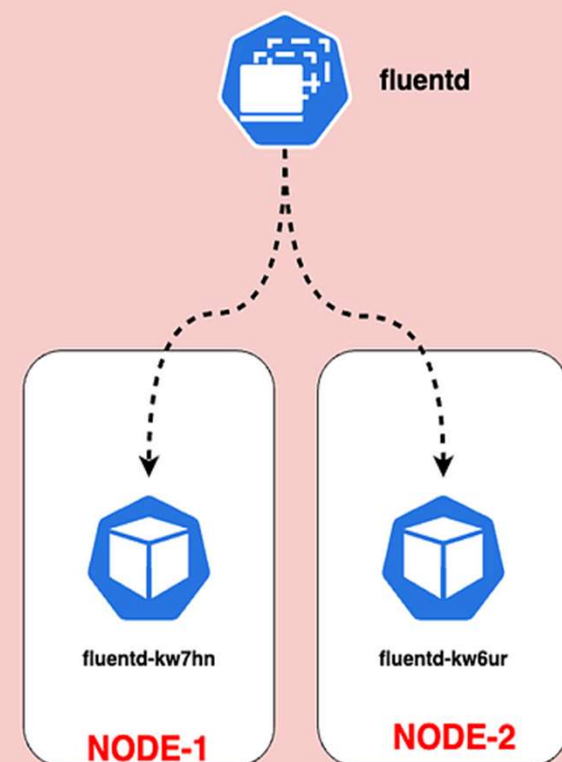
Deployment



StatefulSet



DaemonSet





How DaemonSets Work

1. Write YAML manifest (selector, pod spec, node rules)
2. Apply with `kubectl apply -f ...`
3. Kubernetes schedules pods on matching nodes
 - Auto-updates when nodes are added or removed



Example YAML – All Nodes

apiVersion: apps/v1

kind: DaemonSet

metadata:

name: example-daemonset

spec:

selector:

matchLabels:

name: example-daemonset

template:

metadata:

labels:

name: example-daemonset

spec:

containers:

- name: example-container

image: monitoring-agent



Example YAML – Specific Nodes

...

spec:

template:

spec:

nodeSelector:

example: daemonset

containers:

- image: monitoring-agent



Update Strategies

- **RollingUpdate** (default): auto-updates all pods
- **OnDelete**: manual control—pods only update after deletion
- When to use: real-time systems where avoiding disruption is critical



Common Use Cases

- Log collection & aggregation
- Node observability & metrics gathering
- Security/Intrusion detection
- Network traffic analysis



Best Practices

- Use DaemonSets **only** when needed (node-level apps)
- Prefer **RollingUpdate** to avoid manual overhead
- Monitor resource usage to prevent node overload
- Track configuration changes via Git/version control



Potential Challenges

- **Resource overhead:** one pod per node → high CPU/memory usage
- **Node affinity issues:** mismatched labels cause scheduling failures
- **Node variations:** not all nodes can host same workloads
- **Update complexity:** behavior varies by update strategy, may surprise admins